COMENIUS UNIVERSITY IN BRATISLAVA
FACULTY OF MATHEMATICS PHYSICS AND INFORMATICS

SYNTHESIS OF SOUND FOR APPLICATION IN AN INTERACTIVE GAME FOR VISUALLY IMPAIRED USERS

Master thesis

2025 Bc. Norbert Hašan

COMENIUS UNIVERSITY IN BRATISLAVA
FACULTY OF MATHEMATICS PHYSICS AND INFORMATICS

SYNTHESIS OF SOUND FOR APPLICATION IN AN INTERACTIVE GAME FOR VISUALLY IMPAIRED USERS

Master thesis

Study program:	Applied informatics
Branch of study:	Applied informatics
Department:	Department of Applied Informatics
Supervisor:	Mgr. Lukáš Gajdošech
Consultant:	

Bratislava, 2025

Bc. Norbert Hašan

67751524

Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

ZADANIE ZÁVEREČNEJ PRÁCE

Meno a priezvisko študenta: Bc. Norbert Hašan
Študijný program: aplikovaná informatika (Jednoodborové štúdium, magisterský II. st., denná forma)
Študijný odbor: informatika
Typ záverečnej práce: diplomová
Jazyk záverečnej práce: slovenský
Sekundárny jazyk: anglický

Názov: Syntéza zvuku pre použitie v interaktívnej hre určenej pre ľudí so zrakovým znevýhodnením
Synthesis of sound for application in an interactive game for visually impaired users.

Anotácia: Ľudia so zrakovým znevýhodnením vyžadujú pri hraní interaktívnej hry väčšie množstvo informácie sprostredkovanej pomocou zvuku. Vďaka takémuto výstupu sa vedia napr. zorientovať a vyriešiť priestorový hlavolam. Produkcia zvukovo bohatých prostredí je časovo náročná, vyžaduje zber veľkého počtu zvukových vzoriek a ich následné spracovanie. Toto spracovanie je pritom často pomerne dobre zautomazivateľné, napr. pridanie ekvalizéru, alebo využíva randomizáciu parametrov na vytvorenie viacerých variácií. Príkladom môže byť napr. vytvorenie zvuku pre kroky na rôznom podklade (hlina, piesok, dlažba atď.). Predmetom tejto práce je preskúmanie a návrh rôznych prístupov syntézy zvuky na základe textového popisu dizajnéra. Metódy zahŕňajú analytické ako aj moderné prístupy založené na generatívnych neurónových sieťach.

Cieľ:
- preskúmanie existujúcich metód syntézi zvuku
- konzultácia s cieľovou skupinou používateľov, špecifikácia ich požiadaviek
- vytvorenie modelu na automatickú generáciu zvukových efektov na základe textového popisu

Literatúra:

- MetaSounds. In: Unreal Engine Documentation. https://docs.unrealengine.com/5.3/en-US/metasounds-in-unreal-engine/
- Barahona-Ríos, Adrián and Tom Collins. SpecSinGAN: Sound Effect Variation Synthesis Using Single-Image GANs. In: Proceedings of the 19th Sound & Music Computing Conference, Saint-Étienne, France, 2022.
- Andreu, S., & Villanueva Aylagas, M. Neural Synthesis of Sound Effects Using Flow-Based Deep Generative Models. In: Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment, 18(1), 2-9. 2022. https://doi.org/10.1609/aiide.v18i1.21941

Kľúčové slová: zvuková syntéza, audio hra, generatívne neurónové siete

Vedúci: Mgr. Lukáš Gajdošech
Katedra: FMFI.KAI - Katedra aplikovanej informatiky
Vedúci katedry: doc. RNDr. Tatiana Jajcayová, PhD.

I hereby declare that I have written this thesis by myself, only with help of referenced literature, under the careful supervision of my thesis advisor.

. . . . . .

Bratislava, 2025 Bc. Norbert Hašan

Acknowledgement

Abstract

Keywords:

Abstrakt

Kľúčové slová:

Contents

1 Introduction **1**

2 Motivation **2**

3 Theoretical Background **3**

3.1 Fundamentals of Digital Audio Signal Processing 3

3.1.1 Discrete-Time Signals and Sampling Considerations 3

3.1.2 The Shift to Frequency-Domain Representations 4

3.1.3 Short-Time Fourier Transform (STFT) Configuration 5

3.1.4 Psychoacoustic Compression: The Mel-Scale 5

3.1.5 Dynamic Range and Normalization 6

3.2 Deep Generative Models 6

3.2.1 Taxonomy of Generative Architectures 6

3.2.2 The Generative Adversarial Network (GAN) Framework 8

3.2.3 Conditional Generation and Scaling (BigGAN) 9

3.2.4 Single-Sample Learning (SpecSinGAN) 9

3.3 GAN Stability 10

3.3.1 The Vanishing Gradient Problem 10

3.3.2 The Wasserstein (Earth Mover's) Distance 11

3.3.3 Weight Clipping vs. Gradient Penalty 12

3.3.4 Normalization 13

3.4 Natural Language Conditioning 13

3.4.1 Semantic embeddings 13

3.4.2 The Transformer Architecture 13

3.4.3 Sentence-BERT (SBERT) 14

3.4.4 Integration into the Generative Pipeline 14

3.5 Neural Vocoding 15

3.5.1 The Phase Reconstruction Problem 15

3.5.2 Generative Vocoders (HiFi-GAN) 16

3.6 Evaluation Metrics for Generative Models 16

3.6.1 Inception Score and Fréchet Inception Distance 17

3.6.2 Adapting to Audio 17
3.6.3 Subjective Evaluation 17

4 Existing solutions **19**

5 Work methodology and Research methods **20**

6 Implementation **21**
6.1 Architecture Design 21
6.1.1 The General network (G) 21
6.1.2 The Critic Network (D) and Wasserstein Loss 22
6.1.3 Stabilization via Gradient Penalty 23

7 Thesis research **25**

8 Results and Discussion **26**

List of Figures

3.1 Example of GAN Architecture[14]. 8
3.2 Illustration of WGAN-GP model[2]. 12

List of Tables

Terminology

Chapter 1

Introduction

Chapter 2

Motivation

Chapter 3

Theoretical Background

3.1 Fundamentals of Digital Audio Signal Processing

The implementation of a generative model for environmental sound synthesis—specifically for textures such as footsteps on gravel or knocking on wood—requires a rigorous definition of the data representation. While sound physically exists as continuous air pressure fluctuations, deep generative models require discrete, structured data. This section outlines the signal processing pipeline, justifying the transition from time-domain waveforms to frequency-domain spectrograms by referencing recent advances in neural audio synthesis[3]

3.1.1 Discrete-Time Signals and Sampling Considerations

The first stage of the pipeline is the discretization of continuous audio signals $x(t)$ into discrete sequences $x[n]$. This process is governed by the Nyquist-Shannon Sampling Theorem[12], which dictates that the sampling frequency f_s must be at least twice the maximum frequency component f_{max} of the signal to prevent aliasing artifacts ($f_s \geq 2f_{max}$).

In the domain of neural audio synthesis, the choice of f_s is a critical hyperparameter that balances perceptual fidelity against computational dimensionality.

- **High Fidelity:** Works focused on high-resolution texturing, such as SpecSinGAN, often utilize a standard rate of 44.1 kHz to capture the full spectral brightness of transients.[3]

- **Efficiency:** Conversely, flow-based models like `WaveFlow` (as adapted by Andreu & Aylagas) often downsample to 16 kHz to enable faster iteration and stable training on consumer hardware[1].

For this thesis, sampling rate of 22,050 Hz has been chosen. This creates a "Nyquist ceiling" of approximately 11 kHz. This decision is theoretically grounded in the spectral

characteristics of the target classes: impact sounds (footsteps, knocks) contain the majority of their semantic texture information in the lower 10 kHz band. By halving the standard rate (44.1 kHz), the input dimensionality has been reduced by 50% without suffering the severe high-frequency loss associated with 16 kHz speech-centric models.

3.1.2 The Shift to Frequency-Domain Representations

A pivotal architectural decision in this work is the choice to model audio in the frequency domain (Spectrograms) rather than the time domain (Waveforms). Time-domain modeling has some limitations. One of them being that early generative models are operated directly on raw waveforms $x[n]$. However, raw audio is highly non-stationary and phase-sensitive; a millisecond shift in time fundamentally alters the numerical values of the sample vector, making it difficult for Convolutional Neural Networks to learn invariant features. Furthermore, waveform generation is computationally expensive. As noted by Andreu & Aylagas, while models like WaveFlow can generate high-quality raw audio, they are often slower and harder to control than spectrogram-based approaches[1].

Therefore spectrograms are a new way of thinking that deserves some attention. Transforming audio into a 2D Time-Frequency representation allows us to leverage the massive advancements in Computer Vision generative models. This "Spectrogram-as-Image" paradigm allows the adaptation of high-performance image generation architectures, such as BigGAN[4], to the audio domain. BigGAN demonstrated that scaling up batch sizes and model width in CNNs leads to dramatic improvements in fidelity. By treating audio as an image, we can utilize BigGAN derived techniques such as Spectral Normalization and Orthogonal Regularization to stabilize training[4].

Especially interesting for this thesis are **Phaseless Log-Magnitude Spectrograms**. Research by Barahona-Ríos & Collins (SpecSinGAN) indicates that for non-harmonic, texture-based sounds (like footsteps or pops), phaseless log-magnitude spectrograms often yield better perceptual results than complex-valued representations. They argue that the magnitude contains the structural "texture" of the sound, while the phase can often be adequately reconstructed during inference.

3.1.3 Short-Time Fourier Transform (STFT) Configuration

To generate these spectrograms, we utilize the Short-Time Fourier Transform (STFT). The STFT $X(m, \omega)$ is computed by windowing the discrete signal $x[n]$ and applying the Discrete Fourier Transform (DFT):

$$\mathrm{STFT}\{x[n]\}(m, \omega) = \sum_{n=-\infty}^{\infty} x[n] \cdot w[n-m] \cdot e^{-j\omega n} \tag{3.1}$$

The choice of window size (FFT size) is non-trivial and comes with tradeoffs. A larger FFT size yields better frequency resolution but smears transient events in time—a critical flaw for percussive sounds like "knocking." Conversely, a small FFT size preserves transients but blurs frequency distinctness.

- SpecSinGAN found that an FFT size of 512 produced the most consistent results for impact sound effects[3].
- However, to capture the richer resonance of environmental textures (like the decay of a footstep in a hall), this implementation utilizes a larger FFT size of 1024 with a Hop Length of 256. This overlap (75%) ensures that temporal aliasing is minimized, a strategy also favored in Griffin-Lim phase reconstruction[3].

3.1.4 Psychoacoustic Compression: The Mel-Scale

The linear frequency scale produced by the STFT does not align with human auditory perception, which is logarithmic. To maximize the model's efficiency, we project the linear spectrogram onto the Mel-Scale, compressing the frequency axis to 80 Mel bands. While 80 bands are standard for high-fidelity speech synthesis (e.g., in HiFi-GAN), Andreu & Aylagas note an interesting trade-off: high-dimensional conditioners (like 80-band spectrograms) encourage high quality, but lower-dimensional representations (e.g., 20 or 30 bands) often encourage greater diversity and variation in the generated output.[1]

$$m = 2595 \cdot \log_{10}\left(1 + \frac{f}{700}\right) \tag{3.2}$$

Despite this trade-off, 80 bands should be the most ideal to ensure that the subtle textural differences between surfaces (e.g., "sand" vs. "gravel") are preserved, relying on the Generative Adversarial Network's latent space (z) rather than the spectrogram resolution to introduce diversity.

3.1.5 Dynamic Range and Normalization

Deep neural networks, particularly GANs using Tanh activation functions in the generator, require strictly bounded input data. Audio data, however, has a massive dynamic range. **Logarithmic Scaling** is when the magnitude spectrograms are first converted to Decibels (dB) to compress the dynamic range: $S_{dB} = 20 \log_{10}(S)$. **Normalization:**

- **SpecSinGAN** normalizes spectrograms by subtracting the mean and dividing by the standard deviation[3].
- Andreu & Aylagas, however, normalize waveforms to $(-1, 1)$ and spectrograms to $(0, 1)$[1]
- This thesis To match the Tanh output range of the Generator (which spans $[-1, 1]$), a Min-Max normalization on the dB spectrograms is performed to map them precisely to the range $[-1, 1]$. This alignment is crucial for preventing gradient saturation during the early stages of WGAN-GP training.

This signal processing pipeline effectively transforms the unstructured audio synthesis problem into a structured image generation problem, which opens the possibility to deploy powerful architectures like WGAN-GP and BigGAN to the domain of environmental sound synthesis.

3.2 Deep Generative Models

The synthesis of artificial environmental sounds falls under the domain of Deep Generative Modeling. Unlike traditional discriminative models (such as classifiers) which aim to predict a target label y given an input x by modeling the conditional probability $P(y|x)$, generative models aim to learn the underlying probability distribution $P_{data}(x)$ of the dataset itself. By approximating this distribution, the model can sample new data points x_{new} that are novel, yet statistically indistinguishable from the real training data.

In the context of this thesis, the goal is to model the high-dimensional distribution of audio spectrograms conditioned on textual descriptions. Mathematically, this shifts the objective to learning the conditional distribution $P(x|t)$, where x is the audio spectrogram and t is the text embedding.

3.2.1 Taxonomy of Generative Architectures

Recent advances in deep learning have produced several distinct families of generative models, each with unique theoretical advantages and trade-offs. To justify the selection

of the Generative Adversarial Network (GAN) for this implementation, it is necessary to review the alternative approaches commonly used in neural audio synthesis.

1. **Variational Autoencoders (VAEs)** VAEs operate by encoding input data into a lower-dimensional latent space via a probabilistic encoder $q(z|x)$ and reconstructing it via a deterministic decoder $p(x|z)$. They optimize the Evidence Lower Bound (ELBO) on the log-likelihood of the data. While VAEs offer stable training and a smooth latent space, they are theoretically prone to generating "blurry" outputs because the element-wise reconstruction loss[1], penalizes high-frequency details that are essentially random—such as the fine-grained texture of "gravel" or "sand" sounds.

2. **Flow-Based Models (Normalizing Flows)** Normalizing Flows, such as **WaveFlow**, represent a class of models that learn an explicit, invertible mapping $f : X \to Z$ between the data distribution and a simple prior (usually an Isotropic Gaussian).

   - Unlike GANs, Flow-based models allow for exact log-likelihood maximization. As demonstrated by Andreu & Aylagas (2022) in their work on sound effect synthesis, models like WaveFlow can generate high-quality raw audio waveforms by optimizing the maximum likelihood directly.[1]
   - A unique property of flows is that the generation process is the exact mathematical inverse of the training process. The model learns a function $z = f(x)$ to map data to noise, and generates data by computing $x = f^{-1}(z)$
   - The requirement for the network to be mathematically invertible severely restricts the choice of architectural layers (e.g., standard convolutions are difficult to invert). Furthermore, Flow-based models are computationally expensive for high-dimensional data, often requiring significant memory for the Jacobian determinant computations.[1]

3. **Diffusion Probabilistic Models** Diffusion models have recently emerged as a powerful alternative, operating by gradually adding Gaussian noise to data until it becomes pure noise, and then learning a neural network to reverse this process (denoising). While capable of state-of-the-art sample quality, their iterative sampling process is inherently slow, often requiring hundreds or thousands of network evaluations to generate a single sample[11].

[1] typically Mean Squared Error

3.2.2 The Generative Adversarial Network (GAN) Framework

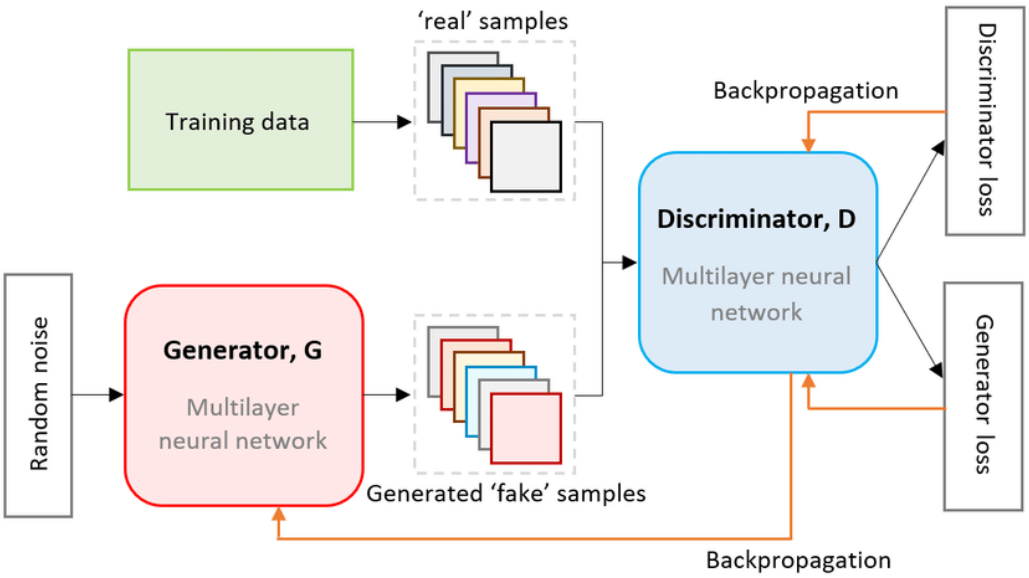

Figure 3.1: Example of GAN Architecture[14].

This thesis constructs its implementation upon the Generative Adversarial Network (GAN) framework, introduced by Goodfellow et al. (2014)[7][14] . GANs differ fundamentally from the methods above: they do not rely on an explicit definition of the probability density function[2] nor do they optimize a lower bound on likelihood[3]. Instead, they utilize a game-theoretic approach to learn the distribution implicitly. It is called **The Minimax Game** where The GAN framework consists of two neural networks competing in a zero-sum minimax game[4]:

1. **The Generator** (G): This network takes a random noise vector z sampled from a prior distribution p_z (typically $N(0, I)$) and maps it to the data space $G(z)$. Its goal is to "fool" the discriminator 3.1.

2. **The Discriminator** (D): This network is a binary classifier that estimates the probability that a given input comes from the real dataset rather than the generator 3.1.

The training process is defined by the following value function $V(D, G)$:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}(x)}[\log D(x)] + \mathbb{E}_{z \sim p_z(z)}[\log(1 - D(G(z)))] \tag{3.3}$$

[2]as in Flows
[3]as in VAEs

Justification for Audio Synthesis GANs are particularly well-suited for environmental sound synthesis for two reasons:

1. The adversarial loss acts as a "texture critic." If the generator produces a spectrogram that is too smooth [4], the discriminator easily identifies it as fake. This forces the generator to synthesize high-frequency spectral details that VAEs would smooth out.

2. Unlike Diffusion models or auto regressive models GANs generate data in a single forward pass, making them highly efficient for real-time applications

3.2.3 Conditional Generation and Scaling (BigGAN)

Standard GANs generate random samples from the dataset. However, for this thesis, the generation must be controlled by a user's textual input. This requires a Conditional GAN (cGAN) architecture, where both the generator and discriminator receive additional information y[5].

The implementation draws significant theoretical inspiration from BigGAN (Brock et al., 2019), which demonstrated that "scaling up" GAN training—increasing batch sizes and channel widths—dramatically improves the fidelity of conditional generation. BigGAN introduced several key concepts utilized in modern conditional synthesis[4]:

- Rather than learning separate class embeddings for every layer, a shared embedding space allows the model to learn semantic relationships between classes (e.g., "dog" and "cat" share fur textures). In our implementation, the Sentence-BERT embedding serves this shared role, projecting semantically similar prompts (e.g., "footsteps on snow" vs "footsteps on sand") into nearby vectors.

- A major discovery in the BigGAN research was that the trade-off between variety and fidelity can be controlled at inference time by truncating the latent distribution z. Sampling z from a narrower range (closer to 0) yields samples with higher fidelity but lower diversity, while sampling from the full normal distribution yields maximum diversity.

3.2.4 Single-Sample Learning (SpecSinGAN)

While BigGAN addresses learning from massive datasets (like ImageNet), environmental sound synthesis often faces data scarcity. A relevant theoretical counterpoint is SpecSinGAN, which adapts the "Single-Image GAN" concept to audio. SpecSinGAN

[4] e.g.,lacking the gritty noise of "gravel"
[5] text embeddings from user)

learns the internal distribution of a single sound effect (e.g., just one recording of a footstep) to generate variations of it.[3] This approach highlights a different generative philosophy being that rather than learning a global semantic space (text-to-audio), SpecSinGAN focuses on "patch-based" learning, capturing the local spectral correlations within a single spectrogram to create new versions.

While implementation of this thesis aims for the broader, multi-class capability of BigGAN-style architectures, understanding the patch-based texture learning of SpecSinGAN is crucial for interpreting how our Convolutional layers process spectral texture.

3.3 GAN Stability

While the Generative Adversarial Network (GAN) framework introduced in 3.2.2 offers a powerful paradigm for implicit distribution learning, its original formulation is notoriously unstable. The mathematical shortcomings of the standard GAN objective must first be analyzed to understand the specific architectural choices made in this thesis to understand the usage of the Wasserstein distance and Gradient Penalty.

3.3.1 The Vanishing Gradient Problem

In the original GAN formulation[7]3.1, the Discriminator D is trained as a binary classifier using the sigmoid cross-entropy loss. If we assume an optimal discriminator D^*, it can be shown that minimizing the Generator's loss is equivalent to minimizing the **Jensen-Shannon Divergence** 3.4 between the real data distribution $\mathbb{P}_r$ and the generated distribution $\mathbb{P}_g$[7] where $\mathbb{M} = \frac{1}{2}(\mathbb{P}_r + \mathbb{P}_g)$ is the mixture distribution.:

$$JS(\mathbb{P}_r||\mathbb{P}_g) = \frac{1}{2}KL(\mathbb{P}_r||\mathbb{M}) + \frac{1}{2}KL(\mathbb{P}_g||\mathbb{M}) \tag{3.4}$$

A critical theoretical issue arises when the supports of $\mathbb{P}_r$ and $\mathbb{P}_g$ are disjoint. In high-dimensional spaces (such as audio spectrograms), two probability distributions are almost certainly disjoint. Arjovsky [2]proved that when the distributions are disjoint, the JS divergence is a constant value ($\log 2$). Consequently, the gradient of the loss function becomes zero:

$$\nabla_\theta JS(\mathbb{P}_r||\mathbb{P}_g) \approx 0 \tag{3.5}$$

This is called the Vanishing Gradient Problem, and it occurs when the discriminator is way too good at separating real and fake audio apart[6], the generator receives no informative feedback on how to improve. Think of a little child trying to learn how to

[6]this is common in early training

play the piano, he can press some keys intuitively, but instead of the teacher being just at the right spot, being praise or especially criticism, the teacher would be too harsh and the kid would never want to move forwards.This frustrates the kid, luckily, our model would just stagnate, but still a negative outcome in the end.

3.3.2 The Wasserstein (Earth Mover's) Distance

To tackle this problem this implementation adopts the Wasserstein GAN (WGAN) framework. The WGAN replaces the JS divergence with the Wasserstein-1 Distance[7]. If we view probability distributions as piles of earth, the Wasserstein distance represents the minimum "work" (mass $\times$ distance) required to transform the generated distribution $\mathbb{P}_g$ into the real distribution $\mathbb{P}_r$. Unlike the JS divergence, the Wasserstein distance provides a smooth, continuous gradient even when the distributions are non-overlapping. Primal form of the Wasserstein distance is intractable to compute directly. However, using the Kantorovich-Rubinstein Duality[2] 3.6, we can express it as:

$$W(\mathbb{P}_r, \mathbb{P}_g) = \sup_{\|f\|_L \leq 1} \mathbb{E}_{x \sim \mathbb{P}_r}[f(x)] - \mathbb{E}_{\tilde{x} \sim \mathbb{P}_g}[f(\tilde{x})] \tag{3.6}$$

Even sooner it has been discussed that, the Discriminator is replaced by a **Critic** function f. The critical condition for this equation to hold is that f must be **1-Lipschitz Continuous**. This means the norm of the gradient of the function must be bounded by 1 everywhere:

$$||\nabla_x f(x)||_2 \leq 1 \tag{3.7}$$

If this constraint is satisfied, the Critic approximates the Wasserstein distance up to a constant factor, providing linear gradients that drive the Generator forwards, since it measures how far the generated data is from the real data, rather than giving a binary 'yes/no' answer that stops providing useful information.

[7]informally known as the Earth Mover's Distance

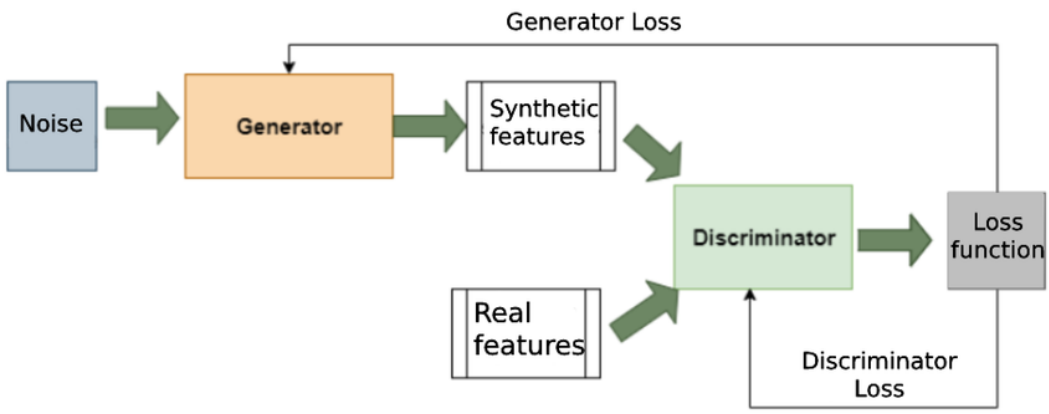

Figure 3.2: Illustration of WGAN-GP model[2].

3.3.3 Weight Clipping vs. Gradient Penalty

The primary challenge in WGAN implementation is strictly obey the Lipschitz constraint. The original WGAN paper proposed Weight Clipping to enforce this constraint: after every gradient update, the weights w of the Critic were clamped to a small box $[-c, c]$. While this ensures boundedness, it leads to pathological optimization behaviors. Clipping forces the network to learn simple functions, limiting its ability to model complex, high-frequency spectral details like the "crunch" of gravel. Also if the clipping parameter c is set incorrectly, gradients either vanish or explode as they propagate through deep networks. To address these limitations, this thesis implements WGAN with Gradient Penalty (WGAN-GP)[9] 3.2. A "soft" constraint by directly penalizing the gradient norm in the loss function is used instead of clipping weights.

Random interpolation points $\hat{x}$ are sampled along the straight lines connecting pairs of real (x_r) and generated (x_g) samples:

$$\hat{x} = \epsilon x_r + (1 - \epsilon) x_g, \quad \epsilon \sim U[0, 1] \tag{3.8}$$

After that the gradient magnitude of the Critic is enforced at these points so they should be close to 1. The final objective function is:

$$L = \underbrace{\mathop{\mathbb{E}}_{\tilde{x} \sim \mathbb{P}_g} [D(\tilde{x})] - \mathop{\mathbb{E}}_{x \sim \mathbb{P}_r} [D(x)]}_{\text{Original WGAN Loss}} + \underbrace{\lambda \mathop{\mathbb{E}}_{\hat{x} \sim \mathbb{P}_{\hat{x}}} \left[(\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1)^2 \right]}_{\text{Gradient Penalty}} \tag{3.9}$$

This formulation should allow the Critic to learn complex functions without side effects of weight clipping, making it ideal for the highly non-linear environmental audio textures.

3.3.4 Normalization

Standard **Batch Normalization** introduces dependencies between samples in a mini-batch (via batch statistics). This invalidates the theoretical derivation of the Gradient Penalty, which assumes that the Critic's output for a sample depends only on that sample. Therefore it is necessary to do some architectural changes in normalization layers. Instance Normalization normalizes each sample independently, preserving the independence required for the valid computation of the gradient penalty term.

3.4 Natural Language Conditioning

Ability to generate audio conditioned on natural language descriptions is one of the novelties of the system. Unlike simple class-conditional models which use a one-hot vector or a learned embedding table for a fixed set of categories, natural language conditioning requires a model capable of interpreting open-ended, unstructured text and projecting it into a dense, continuous semantic vector space.

3.4.1 Semantic embeddings

The more traditional GANs, including the above mentioned architectures of SpecSinGAN and BigGAN typically condition on discrete class labels $y \in \{1, \ldots, K\}$. BigGAN is a bit more advanced in this matter, because the labels are mapped to a learned embedding space where the network can learn correlations between classes[4]. However, class-based conditioning is rigid. It treats "footstep on snow" and "footstep on ice" as entirely distinct orthogonal categories, ignoring their semantic similarity. To enable the synthesis of diverse textures based on descriptive input, we utilize **Semantic Embeddings**. The goal is to map a variable-length sentence $S = \{w_1, w_2, \ldots, w_n\}$ to a fixed-size vector $t \in \mathbb{R}^d$ such that semantically similar sentences lie close together in the vector space[15].

3.4.2 The Transformer Architecture

This thesis utilizes the Transformer architecture[16]. Transformers process the entire sequence in parallel using the **Self-Attention Mechanism**. unlike those Recurrent Neural Networks process text sequentially and therefore struggle with longer range dependencies.

Scaled Dot-Product Attention is very relevant for this solution. For a sequence of input embeddings, the model computes three matrices: Queries (Q), Keys (K), and

Values (V). The attention weights then can be calculated as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{3.10}$$

where d_k is the dimension of the key vectors. This mechanism allows the model to weigh the importance of different words in a sentence relative to each other, capturing context (e.g., how the word "heavy" modifies the acoustic expectation of "boots")

3.4.3 Sentence-BERT (SBERT)

Standard BERT[8] models produce an embedding for every token in the input sentence. To obtain a single fixed-size vector representation for the entire sentence—necessary for conditioning the GAN—we employ Sentence-BERT[15]. Specifically, the implementation utilizes the `all-MiniLM-L6-v2` model. This is lightweight Transformer model designed for efficiency.

1. The raw text prompt (e.g., "knocking on a wooden door") is tokenized into subword units.
2. The 6-layer Transformer processes these tokens, producing context-aware embeddings.
3. SBERT derives the final sentence embedding t by computing the mean of all output token embeddings.

This results in a dense vector of dimension $d = 384$. This vector encapsulates the semantic "texture" of the prompt, allowing the GAN to generalize. For example, because the embedding for "tarmac" is close to "concrete" in this space, the Generator can produce similar impact sounds for both, even if training data for "tarmac" is sparse.

3.4.4 Integration into the Generative Pipeline

The integration of this text embedding into the WGAN-GP architecture follows a strategy of Early Fusion. In the Generator, the text embedding t of dimension 384 is concatenated with the noise vector z of dimension 100 along the channel dimension immediately at the input layer:

$$\text{Input} = [z \parallel t] \in \mathbb{R}^{484} \tag{3.11}$$

This combined vector is then projected via a linear layer to the initial spatial resolution. This approach differs slightly from the conditioning strategy in BigGAN, which

[8] Bidirectional Encoder Representations from Transformers

utilizes "Skip-z" connections to inject the class embedding at multiple resolutions of the network. While the BigGAN approach improves gradient flow for very deep networks (hundreds of layers), this implementation due to technical difficulties relies on that the Early Fusion is sufficient for the Generator depth used (4 blocks) and ensures that the low-level spectral structure is fundamentally shaped by the semantic intent from the very first upsampling operation. In the Critic, a Late Fusion strategy is employed. The text embedding is concatenated with the flattened feature vector after the convolutional feature extraction but before the final classification layer. This forces the Critic to judge whether the generated audio features match the specific text description provided, rather than just judging if the audio sounds realistic in isolation.

3.5 Neural Vocoding

The final stage of the generative pipeline is the conversion of the synthesized Mel-spectrogram back into a time-domain waveform. This process, known as Vocoding[9], is non-trivial because the spectrogram representation is lossy.

3.5.1 The Phase Reconstruction Problem

As established in Section 3.1.3, the Mel-spectrogram consists of the magnitude of the Short-Time Fourier Transform (STFT), typically compressed logarithmically. Mathematically, the STFT of a signal x is a complex number

$$X(m, \omega) = |X(m, \omega)| \cdot e^{j\phi(m,\omega)} \tag{3.12}$$

where $|X|$ is the magnitude (which our GAN generates) and ϕ is the phase (which our GAN discards). To reconstruct the audio, we must estimate this missing phase information.

Historically to do this task **Griffin-Lim** reconstruction was the usual approach. Phase reconstruction was performed using algorithmic heuristics, like the Griffin-Lim algorithm[8]. This iterative method attempts to find the "most likely" signal that corresponds to a given magnitude spectrogram by alternating between time-domain and frequency-domain projections. While SpecSinGAN successfully utilized Griffin-Lim for simple, short impact sounds , previous literature notes that it often introduces characteristic robotic or metallic artifacts. These artifacts arise because Griffin-Lim minimizes the mean squared error of the spectral magnitude but does not enforce perceptual realism, often failing to reconstruct the fine temporal coherence required for realistic environmental textures.

[9] Voice Coding

3.5.2 Generative Vocoders (HiFi-GAN)

To overcome the limitations of algorithmic phase estimation, this thesis employs a Neural Vocoder: a deep generative model trained specifically to map Mel-spectrograms to high-fidelity waveforms. HiFi-GAN (High-Fidelity Generative Adversarial Network) has been chosen, as introduced by Kong and referenced in contemporary audio synthesis research[13]. HiFi-GAN treats vocoding as a supervised translation task, training a generator to produce waveforms that are indistinguishable from real audio when analyzed by a discriminator.

Multi-Period Discrimination HiFi-GAN distinguishes itself from other neural vocoders (like WaveNet) through its discriminator design. Audio signals possess periodic structures (e.g., the repetitive crunch of footsteps or the fundamental frequency of a knock). A standard convolutional discriminator might miss these periodic patterns. HiFi-GAN solves this by employing two types of discriminators:

1. **Multi-Scale Discriminator**: Operates on the raw audio at different downsampling rates to capture global structures and silence/audio transitions

2. **Multi-Period Discriminator**: Reshapes the 1D audio into 2D chunks of varying periods (e.g., every 2nd, 3rd, 5th, 7th sample). This allows the network to explicitly analyze periodic textures, which is crucial for distinguishing between the irregular noise of "gravel" and the resonant thud of "wood.

In this thesis implementation it is ideal to use pre-trained model, to focus more on the goal of the thesis. Furthermore this HiFi-GAN, although originally trained on speech, the generalizability of HiFi-GAN's convolutional filters allows it to reconstruct environmental transients with significantly higher fidelity than Griffin-Lim. By conditioning the generator on the high-resolution (80-band) Mel-spectrograms produced by our WGAN-GP, the vocoder acts as a "texture enhancer," filling in the necessary phase coherence to ensure the generated footsteps sound sharp and physically grounded rather than diffuse or metallic.

3.6 Evaluation Metrics for Generative Models

Evaluating implicit generative models is inherently challenging compared to discriminative tasks, as there is no single "correct" output. The goal is to produce samples that possess both high fidelity (realism) and diversity[10]. To quantify the performance of the WGAN-GP architecture, this thesis employs a combination of objective metrics derived from computer vision and subjective perceptual evaluations

[10] coverage of the data distribution

3.6.1 Inception Score and Fréchet Inception Distance

Standard evaluation metrics for GANs originate in image synthesis. The Inception Score, heavily utilized in the BigGAN experiments, measures two properties: the confidence of the model that an image belongs to a specific classand the diversity of classes generated[4].

However, IS fails to penalize intra-class mode collapse. To address this, the Fréchet Inception Distance (FID) was introduced[4]. FID calculates the Wasserstein-2 distance between the distribution of real and generated images in the feature space of a pre-trained Inception-v3 network:

$$FID = ||\mu_r - \mu_g||^2 + \mathrm{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2}) \tag{3.13}$$

A lower FID indicates that the generated data is statistically closer to the real data distribution. As noted in the BigGAN analysis, FID correlates more robustly with human judgment than Inception Score

3.6.2 Adapting to Audio

Directly applying FID to spectrograms is suboptimal because the Inception network is trained on visual objects, not spectral textures. Consequently, this thesis adopts the Fréchet Audio Distance (FAD)[1]. FAD retains the mathematical formulation of FID but replaces the visual Inception embedding with VGGish, a Convolutional Neural Network pre-trained on the AudioSet corpus. By computing distances in the VGGish latent space, FAD assesses whether the generated audio shares the same background statistics—such as temporal envelopes and frequency distributions—as the reference audio, making it robust to slight time or pitch shifts that would penalize pixel-wise metrics like MSE.

3.6.3 Subjective Evaluation

Despite the utility of objective metrics, quantitative evaluation remains an unsolved problem in audio synthesis. Metrics like FAD do not always align perfectly with human auditory perception. Therefore, the gold standard for evaluation remains the Mean Opinion Score (MOS) test[1]. Following the methodology of SpecSinGAN, human listeners are presented with samples and asked to rate them on a Likert scale based on:

- Does the sample sound like a real physical recording?
- Does the sound align with the text description (e.g., "footsteps on sand")?

While WGAN-GP optimizes the mathematical Wasserstein distance, the final validation of the model's success still relies on these perceptual benchmarks.

Chapter 4

Existing solutions

Chapter 5

Work methodology and Research methods

Chapter 6

Implementation

6.1 Architecture Design

To tackle the primary objective of implementation, Deep Generative Model has been created. This model is capable of doing a synthesis of environmental sounds, such as footsteps on different surfaces. Generation of these sounds is conditioned to a natural language descriptions. Different from speech synthesis, where it is possible to rely on rigid structures for generation. In the task of creating environmental sound the model needs to be capable of capturing the randomness of these sounds, whilst still maintaining semantic coherence with the textual inputs.

To fulfill this objective, this, the system is built upon a Conditional Wasserstein Generative Adversarial Network with Gradient Penalty (cWGAN-GP). The standard GANs are proven, to be very instable when modeling high-dimensional spectral data [10]. This architecture was selected to solve that issue. The implementation leverages **PyTorch** to create **Generator (*G*)** and **Critic**[1] ***D*)**. These two neural networks technically compete against each other, where the Generator creates audio spectrograms from latent noise and text embeddings, and the Critic evaluates the realism of the generated content.

6.1.1 The General network (G)

The generator is the one responsible for mapping a text embedding t and a latent vector z into a Mel-spectrogram x_fake. The architectural design follows a transposed convolutional network[4] structure, effectively acting as a decoder that upsamples low-dimensional latent information into a high-resolution time-frequency representation.

Input Representation and Conditioning The generation process begins with two distinct inputs:

[1]Critic in a more standard GAN is also called the Discriminator.

1. **Text embedding t:** A vector of dimension d_t which has been derived from the sentence-transformer. Semantic intent of the prompt is encapsulated of this embedding.

2. **Latent vector z:** A noise vector of dimension $d_z = 100$ sampled from a standard normal distribution $N(0, 1)$. This component ensures the necessary stochasticity, thanks to which the model can generate diverse variations of the same sound class (e.g., different patterns for "footsteps") without messing-up the semantic meaning.

In the initial stage of the network, these two vectors are concatenated along the channel dimension, resulting in a combined input vector of size 484. This ensures that the entire generation process, from the very first projection, is conditioned on the textual description.

The combined vector is then projected via a fully connected linear layer to a dense representation of shape $(512, 5, 32)$. This tensor is then reshaped to match the format required for 2D convolution ($Channels \times Height \times Width$).
The core of the Generator consists of a stack of four **Transposed Convolutional blocks** `ConvTranspose2d`. Unlike standard interpolation, transposed convolutions contain learnable weights, which allows the network to learn optimal upsampling filters specific to spectral textures. The network then doubles the spatial resolution while halving the channel depth:

- upsamples to 1024 channels in block one
- progressively reduce channels to 256 and 128 blocks two and three
- finalize projection to 1 channel, creates single-channel spectrogram in block four

Rectified Linear Units `ReLU` are employed after every intermediate layer to introduce non-linearity and mitigate the vanishing gradient problem[2]. Crucially, the final output layer utilizes a Hyperbolic Tangent `Tanh` activation function. This function bounds the output values to the range $[-1, 1]$. Because of this, the data pipeline includes a strictly enforced normalization step to scale the logarithmic Mel-spectrograms, from decibels to this interval $[-1, 1]$. This leads to compatibility between the real data distribution and the generator's range.

6.1.2 The Critic Network (D) and Wasserstein Loss

Standard GANs typically use a Discriminator with a Sigmoid output to classify samples as "Real" (1) or "Fake" (0). However, this approach often leads to the vanishing

[2]When a highly accurate Discriminator produces zero gradients, leaving the Generator unable to learn

gradient problem, where the Generator stops learning if the Discriminator becomes too successful too quickly.

To address this issue, **Wasserstein GAN (WGAN)** is implemented. Instead of a binary classifier common for the GAN networks, the network acts as a Critic. It produces a continuous scalar score representing the "realness" of the input, approximating the Earth Mover's (Wasserstein) distance between the real and generated data distributions.

The normalization layers in the Critic were proven to be necessary during the implementation. At first the design utilized **Batch Normalization** `BatchNorm`, a standard technique in Deep Learning. However, theoretical constraints of the WGAN-GP framework dictate that the Critic must satisfy the **1-Lipschitz continuity constraint**. That means the gradient of the Critic must not explode. Batch Normalization creates dependencies between samples in a batch (using batch statistics), which invalidates the independence required for the gradient penalty calculation. The solution seems to be to use **Instance Normalization** `InstanceNorm2d` which normalizes each sample independently using its own statistics, preserving the independence of data points required for the valid computation of the Gradient Penalty.

The Critic mirrors the Generator's structure but in reverse. It utilizes strided 2D Convolutions *`Conv2d` with `stride=2`* to progressively downsample the input spectrogram.

- Activation: LeakyReLU with a negative slope of 0.2 is used. Unlike standard ReLU, LeakyReLU allows a small gradient to flow even for negative values. This helps to prevent "dead neurons" in the Critic [3].

- Late Fusion of Text: Oppposite from the Generator (which fuses text at the start), the Critic injects the text embedding after the convolutional feature extraction. The spectral features are flattened and concatenated with the text embedding before the final dense classification layers. This forces the Critic to first learn clean audio features before evaluating whether those features match the text.

6.1.3 Stabilization via Gradient Penalty

The the Lipschitz constraint that has been mentioned above needs to be strictly enforced. To achieve this the **Gradient Penalty** term is used. It replaces weight clipping method from classic WGAN[2]. Weight clipping quite often leads to weight clustering at the clipping limits which is unwanted behavior. The Gradient Penalty is computed during the training loop by sampling random points along the straight lines between real and fake data pairs.The gradient of the Critic with respect to these interpolated

[3] common failure mode in GAN training

points is calculated, and the loss function is penalized if the norm of this gradient deviates from 1.

The total loss function L for the Critic is thus defined as:

$$L_{critic} = \underbrace{\mathop{\mathbb{E}}_{\tilde{x}\sim\mathbb{P}_g}[D(\tilde{x},t)] - \mathop{\mathbb{E}}_{x\sim\mathbb{P}_r}[D(x,t)]}_{\text{Wasserstein Estimate}} + \underbrace{\lambda \mathop{\mathbb{E}}_{\hat{x}\sim\mathbb{P}_{\hat{x}}}\left[(\|\nabla_{\hat{x}} D(\hat{x},t)\|_2 - 1)^2\right]}_{\text{Gradient Penalty}} \tag{6.1}$$

- $\tilde{x} \sim \mathbb{P}_g$: This represents the Fake Data `fakeSpec` in `train.py`. It is sampled from the generator distribution
- $x \sim \mathbb{P}_r$: This represents the Real Data `realSpec` in `train.p`.
- $\hat{x} \sim \mathbb{P}_{\hat{x}}$: This is the Interpolated Data `interpolated` in `train.py line 52`. It is the random point along the line between the real and fake samples where the gradient is penalized
- t: This is the Text Embedding `textEmb` in `train.py`.
- λ: The penalty coefficient `LAMBDAGP = 20` in `train.py`

Where in the solution λ is set to 20, a hyperparameter that weights the importance of the continuity constraint.

Chapter 7

Thesis research

Chapter 8

Results and Discussion

Conclusion

Bibliography

[1] S. Andreu and M. Villanueva Aylagas. Neural synthesis of sound effects using flow-based deep generative models. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, volume 18, pages 2–9, 2022.

[2] Martin Arjovsky, Soumith Chintala, and Léon Bottou. Wasserstein generative adversarial networks. In *International conference on machine learning*, pages 214–223. PMLR, 2017.

[3] Adrián Barahona-Ríos and Tom Collins. Specsingan: Sound effect variation synthesis using single-image gans. *CoRR*, abs/2110.07311, 2021.

[4] Andrew Brock, Jeff Donahue, and Karen Simonyan. Large scale gan training for high fidelity natural image synthesis, 2019.

[5] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019.

[6] Jesse Engel, Kumar Krishna Agrawal, Shuo Chen, Ishaan Gulrajani, Chris Donahue, and Adam Roberts. GANSynth: Adversarial neural audio synthesis. In *International Conference on Learning Representations*, 2019.

[7] Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial networks, 2014.

[8] D. Griffin and Jae Lim. Signal estimation from modified short-time fourier transform. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 32(2):236–243, 1984.

[9] Ishaan Gulrajani, Faruk Ahmed, Martin Arjovsky, Vincent Dumoulin, and Aaron Courville. Improved training of wasserstein gans, 2017.

[10] Andrew Hennessy, Kenneth Clarke, and Megan Lewis. Generative adversarial network synthesis of hyperspectral vegetation data. *Remote Sensing*, 13(12):2243, 2021.

[11] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models, 2020.

[12] Tero Karras, Miika Aittala, Samuli Laine, Erik Härkönen, Janne Hellsten, Jaakko Lehtinen, and Timo Aila. Alias-free generative adversarial networks, 2021.

[13] Jungil Kong, Jaehyeon Kim, and Jaekyoung Bae. Hifi-gan: Generative adversarial networks for efficient and high fidelity speech synthesis, 2020.

[14] Claire Little, Mark Elliot, Richard Allmendinger, and Sahel Samani. Generative adversarial networks for synthetic data generation: A comparative study, 12 2021.

[15] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks, 2019.

[16] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023.